Tuesday 4/14

Assignment 1 Feedback Notes

- Very synthetic eval - need wide variety of data (e.g. including hashtag), need to include post id?, need to be able to tell it's from HackerNews
    - No handwriting examples
    - Should be fine
- Design decisions should be meaningful
    - Not just different color
    - "Why there's hard-coded 0.8 threshold"
- **Respond to ALL feedback**
- "Weird transcripts" - no summary, need to be actual transcripts
- Abstractions -> coding agents inconsistent with it
    - Need to enforce own sense of abstraction / taste
    - Otherwise mixed results
    - E.g. ghost of unnamed abstractions -> just passing through random strings as dictionary fields -> changing things in code is hard
    - Order / number of stages hard-coded vs. nice abstractions so we can configure / have as many stages as we want
    - Build system rather than just a script
    - Code needs to be well-understood

Course content

- Build app that takes photo of receipt, track interesting trends, can be social - how much different items cost
- Starting from scratch using coding agent - all work / understanding from the coding agent
- Need to learn about the system
- Better way to start:
    - Identify top-level components that coding agent identifies, e.g. front-end products
    - Deciding upfront is hard
    - Build one thing, then see how it connects to other things
    - Important to understand connections / always run tests
    - "This is a design document for a receipt scanning application. It is a web-based application that lets users upload receipts (e.g. images from their phone)
    - "I started writing some design in design.md. This is a brand new project, what are some components we need in order to build this?"
    - Build one component to help understand the core of what we're doing
    - Data structure in the back?
    - "That's a lot of components. I want to first understand receipt parsing and what the data structures look like around that. Let's just focus on a library and CLI component for doing that. Write out future TODOs"
    - Tell it to not read API key
    - Give it the sample of how to use the API key
    - Some models only support chat and not images
    - When Claude says "can rector into a library later" -> should do that

- Claude memory
    - Files stored in home directory not related to project
    - And should be version controlled (diff things for diff projects)
    - CLAUDE.md -> file that goes into root of repo -> general instructions for how to act when running in that directory
    - Always clarify what it is doing
    - User-driven -> don't make changes without the user initiating -> tell it to not nudge you to start building
- General SWE
    - Think very hard before allowing users to upload photos to a server you own
    - Not just going to upload receipts
    - E.g. credit card in the background of a photo
    - Sensitive info
    - Storage adds up when scaling (storing images)
    - Try to not store anything
- Hard to recognize names in receipts just bc of abbreviations
    - Good at numbers
- JSON "parsing" -> should constrain output in a structured way
- Prob just do one thing at a time

Thursday 4/16

- Class demo: web application
- Server on backend with database
- Javascript on frontend
- Storage, and front-end tech
- Storage: sqlite, avoid saving receipt images, just store extracted and user-corrected data
- Front-end: react
- API endpoints for sending requests
- Schema for storage
- pages/content of pages and interactions
- Don't represent currency as floating point
- Store a whole number of cents as an integer
- API: 5 express endpoints
    - scan/upload, list, get, update, delete
- Frontend: three react pages: upload (home), receipt list, editable receipt detail view
- Build order: DB, API, frontend
- Update receipt: patches columns in place, returns updated receipt
    - logical/operational rather than storage
- Maybe don't let users hard delete
- Be weary about "DELETE"
- Receipt update: delete all old items from receipt and insert updates
    - Lots of bad design choices
    - Old receipt items were linked to other db's then it's broken

    - API ever exposed, first request is updating one item, then deletes entire receipt ? bad
    - Be able to update one line item - better API
- add/delete/patch per line item
- Maybe in Claude.md -> treat this as production software at major tech company
    - Take design decisions seriously
    - Rn in class prototype
    - Shift over to production - checks over code design carefully
- Write project spec
- Give some types otherwise they won't know what to write
- Think about what professor's use to design on project specs that we had to fill in
    - Why did we have to fill in certain parts and given parts